Variables and Data Types

Data types

From **text** and **numbers** to **lists**, **people** and **actions**, everything in JavaScript can be represented using data types. Here are the most important ones to we'll be going through to begin with:

Primitive types: These are units that cannot be broken down any further

Data Type	Examples	Purpose
Boolean	`true`, `false`	Answer **yes** or **no** questions
Number	`-10`, `0`, `1`, `100`, `12.5`, `etc`	Represent **numeric** things and do **math**
String	`"Hello there!"`	Represent **text**
Null	`null`	Represent **explicit nothingness**
Undefined	`undefined`	Represent **implicit nothingness**

Data types

Composite types: These are units that can be broken down further

Data Type	Examples	Purpose
Object	`{ name: "Nico", age: 33, hungry: true }`	Represent any object with multiple properties using **keys** and **values**

Array	`[1, 2, 3]`
	`["Buy milk", "Cook dinner", "Sleep"]`
	`[ { name: "Nico", age: 33 }, { name: "Serg", age: 28 }, { name: "Rico", age: 27 } ]`
	`[1, "Julian", { address: ""}]`
		Represent lists of things

Booleans

There's really not much we can do with a boolean by itself, yet, as we will see later on, they are crucial to programming

We usually get **true** or **false** as a result of asking our program a question, like:

5 > 3: Is 5 greater than 3? **true**

5 > 10: Is 5 greater than 10? **false**

We can use these answers to branch our logic. For example, it will allow us to turn these kind of phrases into actionable code:

If the user is older than 18, let them in, otherwise, tell them to come back when they're older.

You can flip a boolean by adding an exclamation mark right before it:

!true becomes **false**
!false becomes **true**

Booleans

Another cool thing that we can do with booleans is **chain them together** in a logical sequence or **ANDs** and **ORs**. In JavaScript:

- **&&** represents **AND**
- **||** represents **OR** *(these are 2 vertical bar characters, not 2 lowercase Ls!)*

Here's are some examples

```
1)    true && true
2)    true && false
3)    true && true && true && false
4)    true || false
5)    false || true
6)    false || true || false
```

***Q:** What will each of those evaluate to?*

Numbers

As you might expect, anything that you can usually do with numbers you can do in JavaScript:

Addition: **10 + 10** gives you back **20**
Subtraction: **10 - 11** gives you back **-1**
Multiplication: **10 * 10** gives you back **100**
Division: **10 / 10** gives you back **1**
Exponentiation: **10 ** 2** gives you back **100** (10 to the power of 2)
Modulos: **12 % 10** gives you back **2** (the remainder of 10 / 2)

Numbers

As you've seen, we can also **compare** numbers to get a boolean back:

Is equal to:	**10 === 10**	gives you back **true**
Is not equal to:	**10 !== 10**	gives you back **false**
Is greater than:	**10 > 10**	gives you back **false**
Is less than:	**10 < 10**	gives you back **false**
Is greater or equal to:	**10 >= 10**	gives you back **true**
Is less or equal to:	**10 <= 10**	gives you back **true**

Finally, you are not limited to a single operation at a time, and you can also group operations together with parenthesis:

1 + 2 * 3	gives you back **7** (multiplication takes precedence)
(1 + 2) * 3	gives you back **9** (with the **()**, we force addition to happen first)

Numbers

Every data type in JavaScript gives you access to some useful **properties**, and number is no exception, although there aren't a lot of them.

To access these properties, you can add a **dot .** in front of the data type. When working with raw numbers, since adding a dot might be interpreted as just trying to add decimals to our number, we need to wrap the number in parenthesis to let JS know our true intent:

```
> (0.12345).toFixed
[Function: toFixed]
```

A couple of useful properties for numbers are **.toFixed()** and **.toString()** try them out in the console!

```
> (0.12345).toFixed(2)
'0.12'
> (0.12345).toString()
'0.12345'
```

Q: What do they do? What type do they each return?

Expressions

All those numbers and calculations we've seen so far are called **expressions**

An **expression** is just a bunch of code that results in a **value** when evaluated:

1 is an expression that results in **1**
1 + 1 is an expression that results in **2**
55 % 10 is an expression that results in **5**
53 % 10 % 2 is an expression that results in **1**
(10 + 10 + 10) * (20 + 20 + 20) is an expression that results in **1800**

Etc...

As you see, an expression can be just a **single number**, or **huge bunch of sub-expressions**. What matters is that you can consider them a **unit of code** that will **give you back a value** when evaluated.

Q: What would happen if you typed **1 2 3** in the console?

Think and have a guess before trying it out, then check to see if your assumption was correct!

Expressions

Now, let's say that I have two calculations that depend on each other. For example:

- Sarah works full-time, and I know that she makes £50,000 a year
- Now, I want to know how much she makes each week
- And then how much she makes per working day

There's 52 weeks in a year, so:

50000 / 52

Cool, that returns **961.5384615384615** (what's with the long number? What would **10 / 3** return?)

Now I want to know how much she makes a day, knowing each week has 5 working days.

How would you do it?

Time for a quick exercise

Solve that problem using the REPL

Variables

If you managed to get the result, you may have thought that the process could have been smoother. You probably either had to:

- Copy and paste the initial answer
- Write the same calculation twice and add the next operation

Q: What would have made it better?

Variables

So far, we've been writing expressions in the console and getting a result back, but we haven't had any way to actually **store the result** of our **expression** so that we could **refer to it later**

As you might have guessed by now, JavaScript, pretty much like almost every other programming language out there, has a mechanism that allows us to to just this: **Variables**

Here's how it works. First let's type:

```
const x = 50000 / 52
```

Now, let's type:

```
x / 5
```

Awesome, so that worked just the same, but we didn't have to repeat ourselves or copy paste stuff!

Now... before we move on, I'd like to tell you just one thing...ready?

NEVER USE VARIABLE NAMES LIKE “X”!!!!

Unless it actually makes sense… Which is… almost NEVER!

We can use pretty much anything for our variable name, as long as it doesn't **start with a number or has any spaces** in between. So don't be shy, make them descriptive!

Variables

Now, if you all just typed variables in completely different ways every time, life would be chaos, so programmers have come up with naming conventions to keep things consistent

Here are some of the popular ones:

snake_case  (hisssssss!)

kebab-case  (this is a yummy Japanese dango, but you get the point)

camelCase  (bumpy!)

While the **Python** community loves their **snake_case** (makes total sense, right?), **99.9999999999%** of JavaScript users stick with **camelCase**, soLetsStickWithThatConventionToo!

Note: One more rule for naming variables is, you are not allowed to use reserved JS keywords because JS already uses those words for other things

Variables

Let's talk a little more about variables:

const is short for **constant**, which means, once a variable has been declared using const, we **won't be able to change it** later on

Should you need to change a variable, there's another keyword you can use instead: **let**

```
const age = 10
const age = 20 // ❌ cannot redeclare a variable
age = 20 // ❌ cannot reassign a constant

let age = 10
let age = 20 // ❌ cannot redeclare a variable
age = 20 // ✅ reassigning a *let* variable is perfectly fine!
```

Q: What kind of **values** do you think we can **store** in a variable?

Variables

So, **any valid JS data type** can be **stored in a variable**. In **some languages**, we are **forced** to **specify the type** of variable we are creating, like:

```
const age: number = 33
const name: string = "Nicolas"
```

However, in plain **JavaScript**, we **don't have to**, in fact, we **can't even if we wanted to**!

Q: Is that a good or a bad thing? Why?

Variables

There are 3 terms to learn when dealing with variables: **declare, initialize,** and **assign**

Declare means to **create a variable** for the first time, using the **const** or **let** keywords

Initialize means to create a variable with an initial value.

Assign means to **give a new value** to an **existing variable**

We can **declare** a variable **without assigning it an initial value**, but this will **only** work with **let**

```
let age // ✅ perfectly valid statement! (declared, but not initialized)
const age // ❌ nope, error (consts must be initialized)
```

Q: What will be the **value** of **age** in the first statement?

Variables

Many times, we want to reassign a value by using the value itself. For example, if we want to increment the value by 10, we can do this:

```
let age = 20
age = age + 10
```

See? We are saying **reassign age** to be the **current value** of age **plus 10**

This is such a common operation, that JS gives us a shortcut for it:

```
age += 10
```

There's also **-=**, ***=**, **/=**, **%=** and ****=**, they all work the same way

Variables

In fact, there's one particular operation that is so common that it got its own operator to make things even shorter:

```
let age = 20
age++
```

And

```
let age = 20
age--
```

Q: What do you think these are doing?

Variables

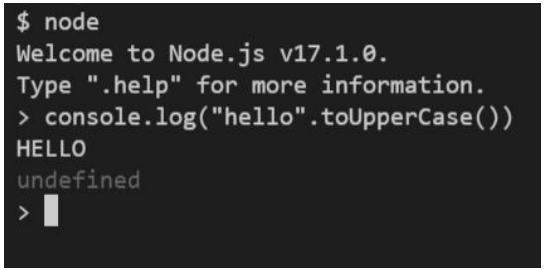
Now you know that we can store values in variables for later use, keep it in mind, because we will be using variables a lot in programming!

Using the REPL

As well as running code from a file, we can use the **REPL** to type and execute code live as we go. The REPL is a **fantastic tool** for learning and experimenting as you start on your JavaScript journey!

REPL stands for Read-Evaluate-Print-Loop. it's an **interactive coding environment** that *Reads* whatever code you provide it with, runs the code (*Evaluates*), *Prints* the result and then *Loops* until you exit.

The REPL is not for writing "serious" programs - it forgets everything as soon as you close it! The REPL is a way to quickly test and run some code if you want to experiment or find out what something does. For longer programs or programs we want to keep, we can run code directly from a file.



Using the REPL

- Start the REPL with the **`node`** command
- Type in JavaScript at the prompt
- Press enter to run!
- The REPL will print out the resulting value of what you typed
- Type **`.exit`** to exit the REPL

```
$ node
Welcome to Node.js v17.1.0.
Type ".help" for more information.
> 10*2
20
> .exit
```